Лабораторная работа № 3

Функции получения системной информации в операционной системе Linux

1. Наименование и цель лабораторной работы

Наименование: «Функции получения системной информации в операционной системе Linux».

Цель работы: получение практических навыков системного программирования на языке C++ в операционной системе Linux, изучение средств получения информации о компьютере, пользователе, операционной системе, системных параметрах, времени и настройках окружения.

2. Постановка задачи

Разработать консольную программу на языке C++ для операционной системы Linux, обеспечивающую получение системной информации и выполнение операций, аналогичных предусмотренным в лабораторной работе для Windows.

В соответствии с вариантом 10 необходимо рассмотреть следующие группы задач:

1. Получение имени компьютера и имени пользователя.
2. Определение путей к основным системным каталогам.
3. Получение сведений о версии операционной системы.
4. Получение трёх системных метрик.
5. Получение трёх системных параметров.
6. Работа с цветами интерфейса.
7. Получение местного времени, информации о часовом поясе и сведений о коррекции системного времени.
8. Проверка состояния клавиатурного ввода, получение сведений о раскладке клавиатуры, определение времени работы системы и работа с региональными настройками.

В отличие от Windows, Linux не предоставляет универсальный аналог Win32 API. Поэтому для выполнения задания используются стандартные функции POSIX, интерфейсы библиотеки C, сведения из файловой системы Linux и Linux-специфичная функция adjtimex.

Для системных метрик выбраны количество доступных процессоров, размер страницы памяти и объём физической памяти. В качестве системных параметров выбраны максимальное количество открытых файловых дескрипторов, максимальное количество дочерних процессов и число тактов системного таймера в секунду. Эти значения выбраны самостоятельно, поскольку методичка не содержит отдельной таблицы системных метрик и параметров для Linux.

Для работы с цветами используется цветовой вывод терминала. Он не изменяет системную цветовую схему графической среды. Для проверки клавиатуры рассматриваются доступные сведения о раскладке и особенности обработки клавиатурного ввода в терминале.

3. Описание методики и алгоритма решения задачи

Программа реализуется на языке C++ с использованием системных интерфейсов Linux. Основная часть системной информации получается посредством библиотечных функций, системных вызовов и чтения стандартных конфигурационных файлов.

Алгоритм работы программы:

1. Подключить заголовочные файлы, необходимые для работы с системными функциями, временем, региональными настройками и файловой системой.
2. Получить имя компьютера с помощью gethostname, имя пользователя — с помощью getpwuid и geteuid.
3. Получить сведения о каталогах /, /usr, /etc и /tmp, а также вывести домашний каталог пользователя, если он указан в окружении.

4. Получить сведения о системе и ядре с помощью uname, а название дистрибутива прочитать из файла /etc/os-release.
5. Получить три системные метрики с помощью sysconf: количество доступных процессоров, размер страницы памяти и количество физических страниц памяти.
6. Получить три системных параметра с помощью sysconf: предел открытых файловых дескрипторов, предел количества дочерних процессов и число тактов системного таймера в секунду.
7. Вывести цветной текст с использованием управляющих последовательностей ANSI. Продемонстрировать временное изменение цвета текста и восстановление стандартного оформления терминала.
8. Получить местное время с помощью time, localtime_r и strftime.
9. Определить сведения о часовом поясе посредством преобразования местного времени и данных, предоставляемых библиотекой C.
10. Получить текущие сведения о коррекции системного времени посредством adjtimex в режиме чтения, не изменяя настройки системных часов.
11. Рассмотреть состояние клавиатурного ввода. Учесть, что стандартный терминал Linux не предоставляет универсальной функции, полностью эквивалентной GetAsyncKeyState.
12. Прочитать сведения о раскладке клавиатуры из доступного конфигурационного файла, если он присутствует в системе.
13. Получить время работы системы с помощью clock_gettime и часов CLOCK_MONOTONIC.
14. Получить текущие региональные настройки времени с помощью setlocale и повторно передать то же значение, не задавая новое.
15. Проверить результаты выполнения системных функций и вывести сообщения об ошибках, если получение информации не удалось.

Изменение системного времени, конфигурации клавиатуры и глобальных настроек локали не выполняется. Это позволяет исследовать соответствующие возможности системы без необходимости менять её настройки.

4. Описание использованных системных функций

gethostname — получает имя компьютера в пределах текущего пространства имён UTS.

getpwuid — получает запись о пользователе по его числовому идентификатору. В программе используется совместно с geteuid, возвращающей эффективный идентификатор пользователя.

uname — получает сведения об операционной системе и ядре, включая имя ядра, имя узла, версию и архитектуру машины.

Чтение /etc/os-release — позволяет получить название и версию дистрибутива Linux, если данный файл присутствует в системе.

sysconf — возвращает значения системных ограничений и параметров. В программе используется для получения количества процессоров, размера страницы памяти, числа физических страниц памяти и некоторых системных пределов.

ANSI escape-последовательности — управляющие последовательности, которые терминал интерпретирует как команды изменения оформления текста. Они позволяют временно изменить цвет выводимого текста и затем вернуть стандартное оформление. Они не изменяют системные цвета окон Linux.

time — получает текущее календарное время в виде значения типа time_t.

localtime_r — преобразует календарное время в структуру tm с учётом текущего часового пояса. В отличие от localtime, эта функция записывает результат в переданную структуру.

strftime — формирует строку даты и времени по заданному формату.

tzset — инициализирует сведения о часовом поясе на основании окружения и доступных системных настроек.

clock_gettime — получает значение указанного системного счётчика времени. В программе применяется с CLOCK_MONOTONIC для определения времени, прошедшего с некоторого момента, не зависящего от изменения календарных часов.

adjtimex — Linux-специфичная функция для получения и настройки параметров коррекции системного времени. В программе она вызывается без задания режима изменения параметров, чтобы получить текущие сведения. Это не является полным аналогом SetSystemTimeAdjustment, поскольку механизмы Windows и Linux различаются.

Чтение /etc/default/keyboard — позволяет получить значение XKBLAYOUT в системах, использующих этот конфигурационный файл. Наличие и актуальность файла зависят от дистрибутива и графической среды.

setlocale — устанавливает или получает региональные настройки для текущего процесса. В программе используется для определения локали времени и повторной передачи уже полученного значения. Эта функция не является аналогом глобального изменения региональных настроек всей системы.

Для ряда функций Windows нет прямого универсального аналога в Linux. В частности, GetAsyncKeyState не имеет общего для всех терминалов интерфейса, позволяющего получить состояние отдельной клавиши в любой момент времени. Также в Linux нет общих для всех графических сред констант, полностью соответствующих COLOR_BTNSHADOW, COLOR_INFOTEXT и COLOR_CAPTIONTEXT.

5. Описание разработанной программы

Разработанная программа является консольным приложением для Linux. Она последовательно получает информацию о системе, пользователе, каталогах, версии ядра и дистрибутива, системных метриках, параметрах, времени и региональных настройках.

Программа выводит результаты в консоль и проверяет успешность выполнения системных функций. Если отдельный конфигурационный файл

отсутствует или функция недоступна, программа сообщает об этом, не подменяя отсутствующие сведения вымышленными значениями.

В качестве аналогии работе с системными цветами демонстрируется цветной вывод текста в терминале. Исходная системная цветовая схема рабочего стола при этом не изменяется. Аналогично, вместо изменения системной коррекции времени программа получает её текущие параметры в режиме чтения.

Программа не требует графического интерфейса. Для компиляции необходим компилятор C++ с поддержкой Linux, например GCC или Clang. Код использует стандартные заголовочные файлы C++ и системные интерфейсы Linux.

6. Листинг разработанной программы

```
#define _GNU_SOURCE

#include <cerrno>
#include <clocale>
#include <cstdint>
#include <cstdlib>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <pwd.h>
#include <string>
#include <ctime>
#include <sys/timex.h>
#include <sys/utsname.h>
#include <unistd.h>

void printSystemValue(const char* name, int parameter)
{
    errno = 0;
    long value = sysconf(parameter);

    std::cout << name << ": ";

    if (value == -1)
    {
        if (errno != 0)
        {
            std::cout << "ошибка получения значения\n";
        }
        else
        {
            std::cout << "значение не определено "
                         "или ограничение отсутствует\n";
        }
    }
    else
    {
        std::cout << value << '\n';
```

```
    }
}

void printOperatingSystem()
{
    struct utsname info{};

    std::cout << "\nСведения об операционной системе:\n";

    if (uname(&info) == 0)
    {
        std::cout << "Ядро: " << info.sysname << '\n';
        std::cout << "Имя узла: " << info.nodename << '\n';
        std::cout << "Версия ядра: " << info.release << '\n';
        std::cout << "Дополнительные сведения: "
                  << info.version << '\n';
        std::cout << "Архитектура: " << info.machine << '\n';
    }
    else
    {
        std::cerr << "Не удалось получить сведения через uname.\n";
    }

    std::ifstream osRelease("/etc/os-release");
    std::string line;
    bool foundName = false;

    if (osRelease)
    {
        while (std::getline(osRelease, line))
        {
            const std::string key = "PRETTY_NAME=";

            if (line.rfind(key, 0) == 0)
            {
                std::string name = line.substr(key.size());

                if (name.size() >= 2 &&
                    ((name.front() == '"' && name.back() == '"') ||
                     (name.front() == '\'' && name.back() == '\'')))
                {
                    name = name.substr(1, name.size() - 2);
                }

                std::cout << "Дистрибутив: " << name << '\n';
                foundName = true;
                break;
            }
        }
    }

    if (!foundName)
    {
        std::cout << "Название дистрибутива: "
                     "не удалось определить из /etc/os-release\n";
    }
}

void printDirectories()
{
    std::cout << "\nОсновные каталоги:\n";
    std::cout << "Корневой каталог: /\n";
    std::cout << "Каталог программ и библиотек: /usr\n";
    std::cout << "Каталог конфигурации: /etc\n";
```

```
    std::cout << "Каталог временных файлов: /tmp\n";

    const char* home = std::getenv("HOME");

    if (home != nullptr)
    {
        std::cout << "Домашний каталог пользователя: "
                  << home << '\n';
    }
    else
    {
        std::cout << "Домашний каталог: "
                     "переменная HOME не задана\n";
    }
}

void printSystemMetrics()
{
    std::cout << "\nСистемные метрики:\n";

    printSystemValue(
        "Количество доступных процессоров",
        _SC_NPROCESSORS_ONLN
    );

    printSystemValue(
        "Размер страницы памяти в байтах",
        _SC_PAGESIZE
    );

    errno = 0;
    long pageSize = sysconf(_SC_PAGESIZE);

    errno = 0;
    long physicalPages = sysconf(_SC_PHYS_PAGES);

    if (pageSize > 0 && physicalPages > 0)
    {
        long double totalMemory =
            static_cast<long double>(pageSize) * physicalPages;

        std::cout << "Объём физической памяти: "
                  << std::fixed << std::setprecision(2)
                  << totalMemory / (1024.0L * 1024.0L)
                  << " МиБ\n";
    }
    else
    {
        std::cout << "Объём физической памяти: "
                     "не удалось определить через sysconf\n";
    }
}

void printSystemParameters()
{
    std::cout << "\nСистемные параметры:\n";

    printSystemValue(
        "Максимальное число открытых файловых дескрипторов",
        _SC_OPEN_MAX
    );

    printSystemValue(
        "Максимальное число дочерних процессов",
```

```
        _SC_CHILD_MAX
    );

    printSystemValue(
        "Число тактов системного таймера в секунду",
        _SC_CLK_TCK
    );
}

void demonstrateTerminalColors()
{
    std::cout << "\nЦветной вывод в терминале:\n";

    std::cout
        << "Аналог тени кнопок: \033[90mсерый текст\033[0m\n";

    std::cout
        << "Аналог информационного текста: "
        << "\033[33mжёлтый текст\033[0m\n";

    std::cout
        << "Аналог текста заголовка: "
        << "\033[1;37mсветлый текст\033[0m\n";

    std::cout
        << "Временное изменение цвета: "
        << "\033[31mкрасный текст\033[0m\n";

    std::cout
        << "Стандартное оформление терминала восстановлено "
        << "последовательностью сброса.\n";

    std::cout
        << "Примечание: цвета графической среды не изменяются.\n";
}

void printLocalTime()
{
    std::cout << "\nДата, время и часовой пояс:\n";

    std::time_t currentTime = std::time(nullptr);
    std::tm localTime{};

    if (currentTime == static_cast<std::time_t>(-1) ||
        localtime_r(&currentTime, &localTime) == nullptr)
    {
        std::cerr << "Не удалось получить местное время.\n";
        return;
    }

    char timeBuffer[128]{};

    if (std::strftime(
            timeBuffer,
            sizeof(timeBuffer),
            "%d.%m.%Y %H:%M:%S",
            &localTime) > 0)
    {
        std::cout << "Местные дата и время: "
                  << timeBuffer << '\n';
    }
    else
    {
        std::cout << "Не удалось отформатировать дату и время.\n";
```

```
    }

    tzset();

    if (localTime.tm_zone != nullptr)
    {
        std::cout << "Обозначение часового пояса: "
                  << localTime.tm_zone << '\n';
    }

    std::cout << "Смещение относительно UTC в секундах: "
              << localTime.tm_gmtoff << '\n';
}

void printTimeAdjustment()
{
    std::cout << "\nКоррекция системного времени:\n";

    struct timex info{};

    // Нулевой режим используется для чтения текущих параметров.
    int result = adjtimex(&info);

    if (result == -1)
    {
        std::cerr << "Не удалось получить параметры adjtimex. "
                  << "Код ошибки: " << errno << '\n';
        return;
    }

    std::cout << "Состояние синхронизации времени: "
              << result << '\n';

    std::cout << "Смещение времени, мкс: "
              << info.offset << '\n';

    std::cout << "Частотная коррекция, ppm: "
              << static_cast<double>(info.freq) / 65536.0
              << '\n';

    std::cout << "Параметры только прочитаны; "
                 "системное время не изменялось.\n";
}

void printKeyboardInformation()
{
    std::cout << "\nСведения о клавиатуре:\n";

    std::ifstream keyboardFile("/etc/default/keyboard");
    std::string line;
    bool foundLayout = false;

    if (keyboardFile)
    {
        while (std::getline(keyboardFile, line))
        {
            const std::string key = "XKBLAYOUT=";

            if (line.rfind(key, 0) == 0)
            {
                std::cout << "Настройка XKBLAYOUT: "
                          << line.substr(key.size()) << '\n';
                foundLayout = true;
            }
```

```
        }
    }

    if (!foundLayout)
    {
        std::cout << "Сведения о раскладке не найдены "
                     "в /etc/default/keyboard.\n";
    }

    std::cout
        << "Проверка состояния отдельной клавиши: "
        << "универсального аналога GetAsyncKeyState "
        << "для всех терминалов нет.\n";
}

void printUptime()
{
    std::cout << "\nВремя работы системы:\n";

    struct timespec uptime{};

    if (clock_gettime(CLOCK_MONOTONIC, &uptime) == 0)
    {
        std::cout << "Прошло секунд: "
                  << uptime.tv_sec << '\n';

        std::cout << "Дополнительные наносекунды: "
                  << uptime.tv_nsec << '\n';
    }
    else
    {
        std::cerr << "Не удалось получить CLOCK_MONOTONIC.\n";
    }
}

void printLocaleInformation()
{
    std::cout << "\nРегиональные настройки:\n";

    const char* currentLocale = std::setlocale(LC_TIME, "");

    if (currentLocale == nullptr)
    {
        std::cout << "Не удалось определить локаль времени.\n";
        return;
    }

    const std::string savedLocale(currentLocale);

    std::cout << "Текущая локаль времени процесса: "
              << savedLocale << '\n';

    if (std::setlocale(LC_TIME, savedLocale.c_str()) != nullptr)
    {
        std::cout
            << "Текущее значение локали повторно передано "
            << "без намеренного изменения.\n";
    }
    else
    {
        std::cout << "Не удалось повторно установить локаль.\n";
    }

    std::cout
```

```
        << "Изменяются только настройки текущего процесса, "
        << "а не глобальная локаль системы.\n";
}

int main()
{
    std::setlocale(LC_ALL, "");

    std::cout
        << "Лабораторная работа № 1\n"
        << "Функции получения системной информации в Linux\n"
        << "Адаптация варианта 10\n\n";

    char hostname[256]{};

    if (gethostname(hostname, sizeof(hostname)) == 0)
    {
        hostname[sizeof(hostname) - 1] = '\0';
        std::cout << "Имя компьютера: " << hostname << '\n';
    }
    else
    {
        std::cerr << "Не удалось получить имя компьютера.\n";
    }

    passwd* user = getpwuid(geteuid());

    if (user != nullptr && user->pw_name != nullptr)
    {
        std::cout << "Имя пользователя: "
                  << user->pw_name << '\n';
    }
    else
    {
        std::cout << "Не удалось получить имя пользователя.\n";
    }

    printDirectories();
    printOperatingSystem();
    printSystemMetrics();
    printSystemParameters();
    demonstrateTerminalColors();
    printLocalTime();
    printTimeAdjustment();
    printKeyboardInformation();
    printUptime();
    printLocaleInformation();

    std::cout << "\nРабота программы завершена.\n";

    return 0;
}
```

7. Результаты выполнения программы

Результаты работы программы представлены в табличной форме. Конкретные значения зависят от используемого дистрибутива Linux, конфигурации оборудования, настроек пользователя и параметров ядра. Поэтому в таблицах

приводятся сведения, которые программа получает при запуске, а не заранее заданные значения.

Таблица 1 — Получение основных системных сведений

Проверяемый параметр	Используемый интерфейс	Результат
Имя компьютера	gethostname	Имя узла системы
Имя пользователя	getpwuid, geteuid	Имя пользователя текущего процесса
Корневой каталог	/	Корневой каталог файловой системы
Каталог программ и библиотек	/usr	Каталог программ и библиотек
Каталог конфигурации	/etc	Каталог системных конфигурационных файлов
Каталог временных файлов	/tmp	Каталог временных файлов
Версия ядра	uname	Название и версия ядра
Название дистрибутива	/etc/os-release	Название дистрибутива, если сведения доступны

Таблица 2 — Системные метрики

Метрика	Используемая функция	Результат
Количество доступных процессоров	sysconf(_SC_NPROCESSORS_ONLN)	Число доступных процессоров
Размер страницы памяти	sysconf(_SC_PAGESIZE)	Размер страницы в байтах

Объём физической памяти	sysconf(_SC_PHYS_PAGES) и sysconf(_SC_PAGESIZE)	Расчётный объём физической памяти

Таблица 3 — Системные параметры

Параметр	Используемая функция	Результат
Максимальное число открытых файловых дескрипторов	sysconf(_SC_OPEN_MAX)	Системное ограничение
Максимальное число дочерних процессов	sysconf(_SC_CHILD_MAX)	Системное ограничение, если оно определено
Число тактов системного таймера в секунду	sysconf(_SC_CLK_TCK)	Значение системного параметра

Таблица 4 — Работа с цветами

Обозначение в исходном задании	Реализация в Linux	Результат
COLOR_BTNSHADOW	Серый текст терминала	Демонстрация близкого по смыслу цвета
COLOR_INFOTEXT	Жёлтый текст терминала	Демонстрация выделенного информационного текста
COLOR_CAPTIONTEXT	Светлый текст терминала	Демонстрация выделенного текста заголовка
Изменение системного цвета	ANSI-последовательности	Временное изменение цвета выводимого текста с

		последующим сбросом оформления

Использованные цвета являются условными аналогами. В Linux отсутствуют универсальные системные цветовые константы, полностью соответствующие перечисленным константам Windows. Цветовая схема графического окружения не изменяется.

Таблица 5 — Работа с временем

Функция или интерфейс	Получаемые сведения
time и localtime_r	Текущие местные дата и время
strftime	Строковое представление даты и времени
tzset	Инициализация сведений о часовом поясе
adjtimex	Текущие параметры коррекции системного времени в режиме чтения
clock_gettime(CLOCK_MONOTONIC)	Время, прошедшее с начала отсчёта монотонных часов

adjtimex используется для получения параметров коррекции системного времени. Программа не изменяет системные часы и не задаёт новые параметры коррекции. Это безопасная адаптация задачи, поскольку механизмы коррекции времени в Linux отличаются от механизма SetSystemTimeAdjustment в Windows.

Таблица 6 — Дополнительные функции варианта 10

Функция Windows	Аналог или адаптация в Linux	Результат

GetAsyncKeyState	Универсального аналога для всех терминалов нет	Программа сообщает об ограничении
GetKeyboardLayoutList	Чтение XKBLAYOUT из /etc/default/keyboard	Получение настроенной раскладки, если файл доступен
GetTickCount	clock_gettime(CLOCK_MONOTONIC)	Получение времени монотонного счётчика
SetLocaleInfo	setlocale	Получение и повторная установка локали времени в текущем процессе

Таким образом, в программе реализованы доступные для Linux способы получения системной информации. Функции, не имеющие универсального аналога в Linux, обозначены отдельно, а их возможности не подменяются утверждением о полном соответствии интерфейсам Windows.

8. Заключение

В ходе лабораторной работы была разработана программа на языке C++ для операционной системы Linux, предназначенная для получения системной

информации с использованием системных интерфейсов и стандартных библиотечных функций.

В программе реализованы получение имени компьютера и пользователя, сведений об операционной системе и ядре, определение системных метрик и параметров, вывод цветного текста в терминале, получение местного времени и информации о часовом поясе, чтение параметров коррекции системного времени, получение сведений о раскладке клавиатуры и работа с региональными настройками процесса.

В результате работы были изучены основные способы взаимодействия прикладной программы с операционной системой Linux, получения системных параметров, обработки ошибок и использования интерфейсов времени.

При адаптации задания были учтены различия между Windows и Linux. Установлено, что отдельные возможности Win32 API не имеют прямого универсального аналога в Linux и требуют использования иных интерфейсов или ограничения функциональности.